Szoftverfejlesztő és -tesztelő szakma

5 0613 12 03

B) vizsgarész: Asztali- és webes szoftverfejlesztés, adatbázis-kezelés

Vizsga időpontja: 2025. május 20.
Rendelkezésre álló idő: 240 perc

Szakmai Bevezető: Informatikai Megoldások az Állatgyógyászatban

Az állatgyógyászat az elmúlt évtizedekben látványos fejlődésen ment keresztül. Napjainkban egy modern állatkórház vagy menhely működése már elképzelhetetlen precíz, digitális adatnyilvántartás és automatizált folyamatok nélkül. Legyen szó a páciensek (kutyák, macskák, haszonállatok) törzsadatainak kezeléséről, az oltási rendszerek nyomon követéséről vagy a sürgősségi diagnosztikáról, a szoftveres támogatás alapvető elvárás. Az informatikai rendszereknek nem csupán tárolniuk kell az adatokat, hanem intelligens módon segíteniük kell az orvosok és gondozók döntéshozatalát – például az etetési költségek kalkulálásával vagy az örökbeadhatósági státusz automatikus kiértékelésével.

Az állatgyógyászati szoftverfejlesztés során a legnagyobb kihívást az élővilág sokszínűségének leképezése jelenti. Ahogy a biológiában, úgy a programozásban is rendszertanra van szükség: közös tulajdonságokat és egyedi viselkedésformákat kell definiálnunk. Itt válik elengedhetetlenné az objektumorientált szemléletmód (OOP), amely lehetővé teszi, hogy a való világ entitásait hatékony és újra felhasználható kódszerkezetekbe öntsük.

A vizsga célja és felépítése

Ebben a vizsgafeladatsorban egy komplex állategészségügyi és menhelykezelő keretrendszer alapjait kell lefektetnie. A feladatok során az állatgyógyászat logikáját követve kell alkalmaznia a C# nyelv haladó eszköztárát.

Absztrakció és Interfészek. Öröklődés és Polimorfizmus. Adatkezelés és API integráció. Felhasználói élmény (WPF). Weboldal keretrendszer segítségével (React) esetleg statikus weboldal.

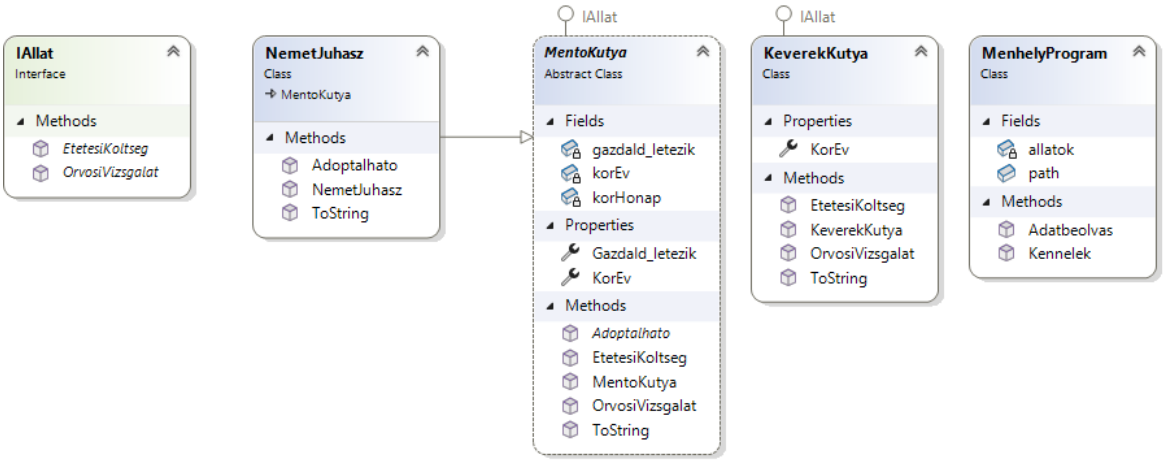
A következő feladatok megoldása során törekedjen a tiszta kód elveinek betartására és a logikus mappaszerkezet kialakítására. Gondoljon úgy a kódjára, mint egy digitális praxis alapkövére, ahol minden osztály és metódus az állatok hatékonyabb ellátását szolgálja.

Nevezze át a Forrás nevű mappát a saját nevére a H: meghajtó gyökér könyvtárában. Abban megtalálja a megfelelő feladatrészhez tartozó projekteket, állományokat, rendszereket.

Jó munkát!

Konzolos asztali alkalmazásfejlesztés – 15 pont

Készítsen egy projektet az „Asztali console” nevű mappába **KutyaMenhely_console** néven, és oldja meg az alábbi feladatokat! A mappa tartalmazza az adatok.txt állományt, aminek az első sora a mezőnevek és az adatok pontosvesszővel vannak elválasztva.



1. Írjon egy **interfészt** *IAllat* néven, amely a következőket tartalmazza:
   - Egy logikai *OrvosiVizsgalat* nevű metódust, amely eldönti, hogy az állat állapota megfelelő-e.
   - Egy *EtetesiKoltseg* nevű metódust, amely paraméter nélkül visszaadja az állat napi etetési költségét lebegőpontos értékként.
2. Írjon *MentoKutya* néven egy **absztrakt** osztályt, amely implementálja az **IAllat** interfészt.
   - A mentőkutyákat jellemezzék a következő osztály szintű mezők/adattgok:
     - *korEv* (egész),
     - *korHonap* (egész)
     - *gazdaId_letezik* (logikai)
   - Az osztály példányosítása során paraméteren keresztül kapjanak értéket a korábbi mezők.
   - Legyen egy osztályon kívül, bárhol látható **abstract** metódusa *Adoptalhato* néven, amely logikai értékkel tér vissza.
   - Az örökölt metódusok implementálása:
     - Az *EtetesiKoltseg*() számítása:
       alapdíj (500 egység) + a kutya éveinek száma * 100.

    - Az *OrvosiVizsgalat* () akkor térjen vissza igazzal, ha a kutya 10 évnél fiatalabb.
- **Property-k:**
    - A *gazdaId_letezik* értéke csak olvasható legyen.
    - A *korEv* értéke bárhonnan lekérdezhető legyen, de módosítani csak az osztályban és a származtatott osztályokban lehessen.
- **Szöveges forma:** Definiálja felül a ToString metódust, hogy írja ki az életkort évben és hónapban majd egy másik, alatta lévő sorban tabulátorral becsúsztatva az etetési költséget.

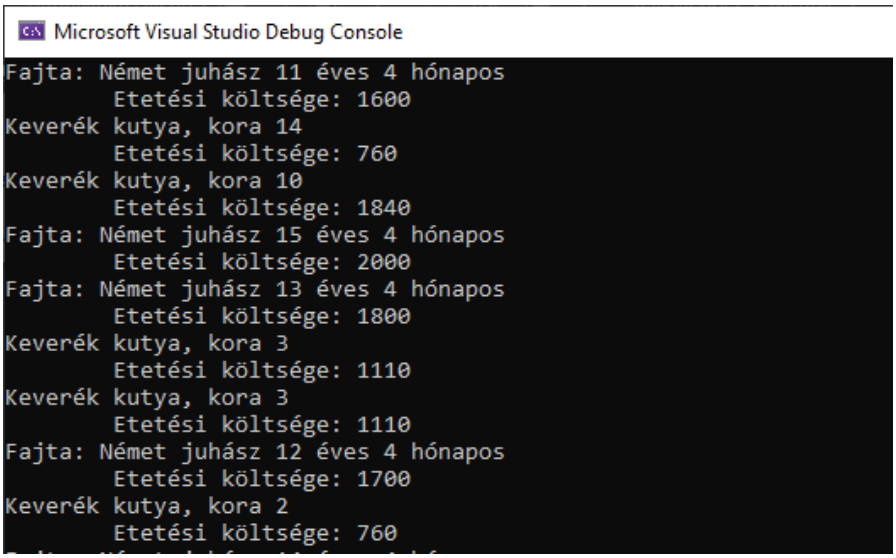
3. Írjon egy *NemetJuhasz* osztályt, amely a **MentoKutya** leszármazottja:
    - Állítsa be alapértelmezettnek, hogy példányosítás során, ha nem adnak meg adatot, akkor a fajtára jellemző adatokat: 2 év, 4 hónap, és kezdetben nincs gazdája.
    - **Adoptalhato metódus:** Egy német juhász akkor adoptálható, ha nincs gazdája ÉS az orvosi vizsgálata rendben van.
    - **Szöveges forma:** Egészítse ki az ősosztály ToString kimenetét a "Fajta: Német juhász" szöveggel. (Lásd majd a mintánál)
4. Írjon egy KeverekKutya osztályt, amely implementálja az IAllat interfészt:
    - Legyen egy publikusan lekérhető, de csak osztályszinten beállítható KorEv tulajdonsága.
    - Rendelkezzen paraméteres konstruktorral, amely beállítja a kutya életkorát.
    - **Metódusok:**
        - Az etetési költsége a 2000 * sin(korÉv*PI/16).
        - Az orvosi vizsgálat minden esetben igazat ad vissza.
    - **Szöveges forma:** Írja ki, hogy "Keverék kutya, kora: 5" alatta lévő sorban tabulátorral behúzva „ Etetési költség: 480”. Az etetési költség 10-esekre kerekítve jelenjen meg! (Lásd majd a mintánál)
5. Írjon egy Menhely nevű osztályt, benne a statikus metódusokkal:
    - **osztályszintű AdatBeolvasas(string path):** Olvassa be az *adatok.txt* fájlt. Hozzon létre NemetJuhasz vagy KeverekKutya példányokat. Hibás fájl vagy futás esetén ne álljon le a program (try,catch). Kivétel esetén adjon vissza egy nem található fájl kivételt (FileNotFoundException) Tárolja az objektumokat egy közös gyűjteményben majd a beolvasás során térjen vissza ezzel.

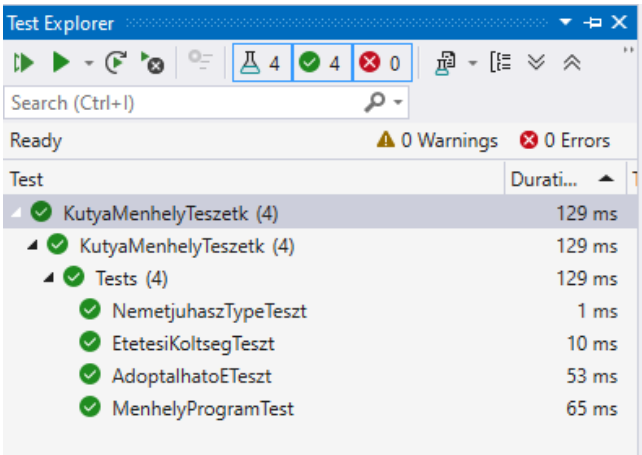
- osztályszintű allatok nevű lista, ami megkapja az AdatBeolvasas(path) eredményét!
- **Kennelek metódus:** Menjen végig az allatok listán és írja ki minden kutya adatait a konzolra a hozzá tartozó etetési költséggel. Figyeljen rá, ha olvasási hiba (Kivételkezeléssel oldja meg)

6. A program működését ellenőrizze a Program osztály Main metódusában. Írassa ki a Menhely osztály Kennelek függvény eredményét!

Használható dokumentációk:

https://learn.microsoft.com/en-us/dotnet/csharp/





Grafikus alkalmazásfejlesztés – 10 pont

Nyissa meg az **Asztali GUI (WPF)/Allatgyogyaszat_GUI** wpf alkalmazást. A grafikai rész készen. Új eszközt nem kell használnia. De ha a meglévő eszközökkel nem tudja a feladatot megoldani, használhat új eszközöket. Ez esetben csak részpont adható a feladatra.

A projekt használ egy **Models**, egy **Services** és egy **VieModels** mappát, ahol előre megvannak írva az API eléréséhez szükséges metódusok. Ezeket szükség szerint használja a megoldása során!

Futtatás során az adatok el vannak helyezve a *ListBox* komponensbe. Illetve van egy Adatok Frissítése gomb, ami az új adat feltöltése vagy módosítás után használható! Figyeljen a feladatok megfogalmazására és a pontos megoldásra! A feladatok nem írják le a pontos esemény nevét, de egyértelműen jelzik mit kell használni.

1. Adott egy *textbox*, ahol biztosítani kell a név alapján a keresést.
    a. Egészítse ki a programot olyan eseménnyel, hogy amikor a felhasználó begépeli a szöveget, egyből szűrje a *listboxban* az adatokat név alapján!
    b. A begépelt szöveg, mindig a név elején nézi az egyezést!
    c. Hosszú nevek beírásánál is működnie kell a programnak!
    d. Ha a *textbox* üres, akkor jelenítse meg az összes adatot!
    e. A szűrésnek úgy is kell működnie, hogy a felhasználó kis betűkkel írja be a nevet!
2. Adott egy *combobox* (legördülő lista),
    a. amibe szűrje ki az adatokból a program indításánál, milyen kerületek vannak a listában! (Ha ez nem megy, akkor csak helyezzen el két értéket pl. 10,20)
    b. Mindegyik kerületből csak egyet jelenítsen meg!
    c. A kerületek csökkenő sorrendben szerepeljenek!
3. Írjon olyan eseményt, ami biztosítja azt, hogy ha a legördülő listából kiválasztunk egy kerületet, akkor a hozzá tartozó adatok jelennek meg, ezzel a kerülettel!
    a. Ha még nem lett kiválasztva adat a legördülő listából, akkor jelenjen meg az összes adat!
4. Az 1. és 2. feladatnak NEM KELL egyszerre működnie! A két szűrés elég, ha külön működik! Ehhez állítsa be, hogy
    a. ha a *textbox*-ba írunk legyen üres a *combobox*,
    b. ha a *comboboxból* választunk akkor legyen üres a *textbox*.

5. Látható az ablakon egy *groupbox*.
    a. Ez kezdetben ne legyen látható!
    b. Ha a *checkbox*-ra rákattintunk akkor váljon láthatóvá!
    c. Ha a *checkbox*-ból kiszedjük a pipát, akkor ennek megfelelően legyen láthatatlan a *groupbox*!
6. Az Új adat felvétele gombra kattintva lehessen felvinni adatot az adatbázisba!
    a. A *textboxba* lehessen az adatokat pontosvesszővel megadni! (pl. „Kis Pista;+36123456789;LittleP@kukac.hu;5” azaz Név;telefonszám;email;kerület)
    b. Majd kattintásra kerüljön be az adatbázisba az itt megadott adatsor!
    c. Feltöltés esetén írassa ki a minta alapján: „Sikeres adatfeltöltés!” vagy ha nem jön vissza sikeres válasz az apitól, akkor „Sikertelen adatfeltöltés!”
    d. Mindkét esetben jelenjen meg az ablakon a „Válasz” felirat!
7. Ha a listából kiválasztunk egy gazdát, akkor duplakattintás hatására nyíljon meg a *GazdaWindow* egy példánya!
    a. Módosítsa úgy a *GazdaWindow*-ot, hogy tudjon megnyitáskor adatot fogadni!
    b. A megérkezett gazda adatai jelenjenek meg a megfelelő helyen az ablakban!
8. A Módosítás gombra kattintva a megadott adat módosuljon az adatbázisban! (Figyeljen rá, hogy a módosításnál az adat felküldésénél szükséges az Id megadása is!)
    a. Feltöltés esetén írassa ki a minta alapján: „Sikeres adatfeltöltés!” vagy ha nem jön vissza sikeres válasz az apitól, akkor „Sikertelen adatfeltöltés!”
    b. Mindkét esetben jelenjen meg az ablakon a „Válasz” felirat!
    c. Minden esetben zárja be ezt az ablakot a Módosítás végén!

Használható dokumentációk:

https://learn.microsoft.com/en-us/dotnet/csharp/

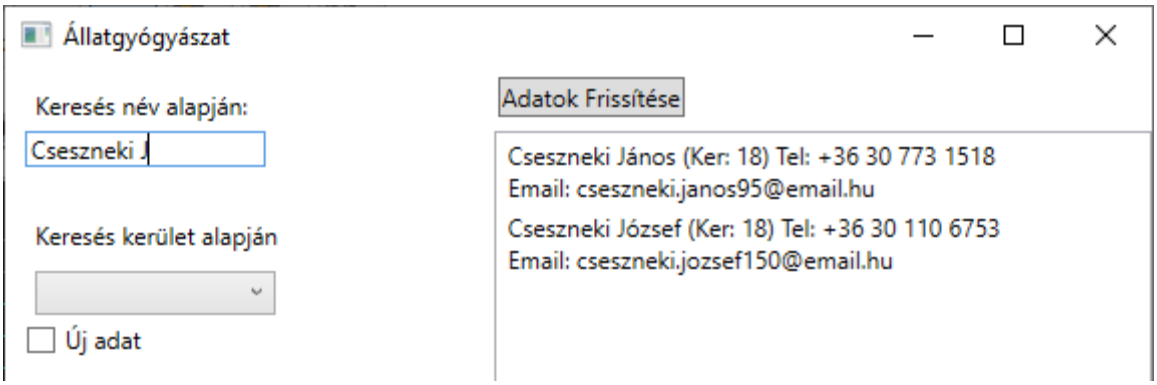
https://learn.microsoft.com/en-us/dotnet/desktop/wpf/

https://learn.microsoft.com/en-us/dotnet/csharp/linq/

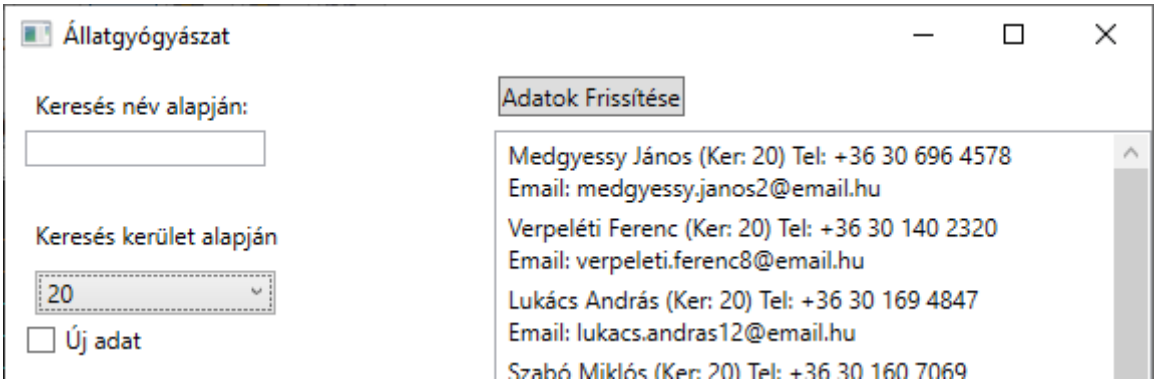
Minta:

Kezdő állapot:

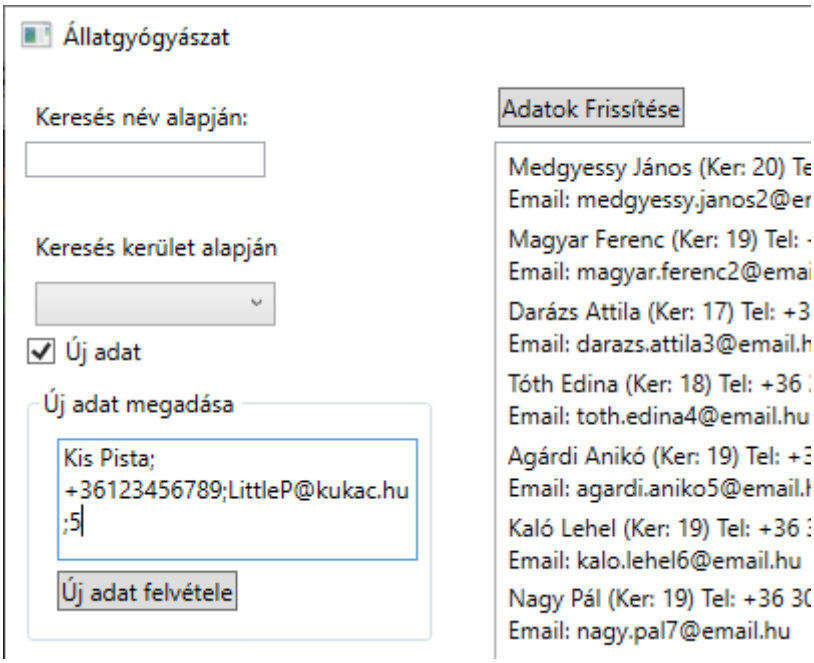
Keresés név alapján

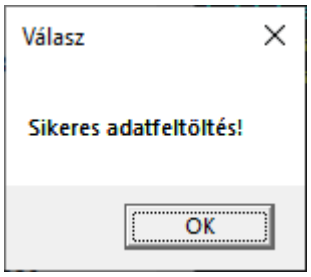


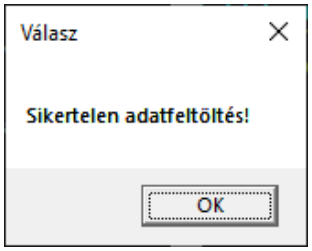
Keresés kerület alapján

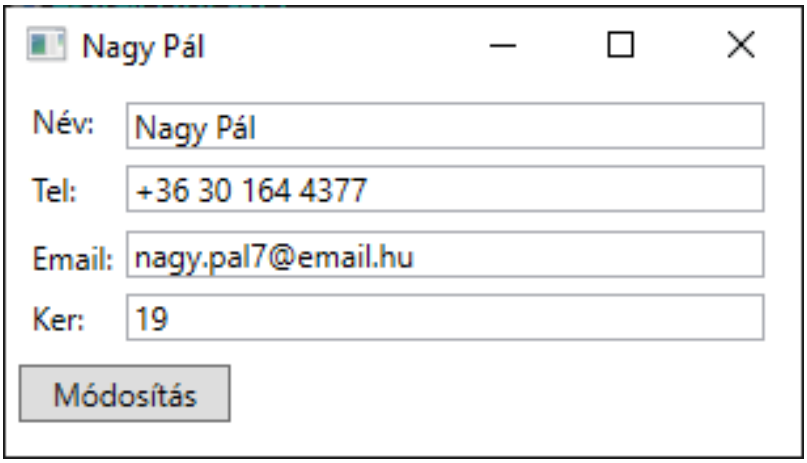


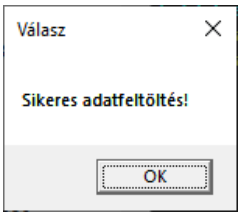
Új adat checkbox:

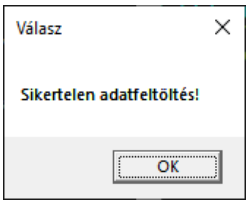

Állatgyógyászat
Keresés név alapján:
Keresés kerület alapján
Új adat
Új adat megadása
Kis Pista;
+36123456789;LittleP@kukac.hu
;5
Új adat felvétele
Adatok Frissítése
Medgyessy János (Ker: 20) Te
Email: medgyessy.janos2@er
Magyar Ferenc (Ker: 19) Tel: -
Email: magyar.ferenc2@emai
Darázs Attila (Ker: 17) Tel: +3
Email: darazs.attila3@email.h
Tóth Edina (Ker: 18) Tel: +36
Email: toth.edina4@email.hu
Agárdi Anikó (Ker: 19) Tel: +3
Email: agardi.aniko5@email.h
Kaló Lehel (Ker: 19) Tel: +36
Email: kalo.lehel6@email.hu
Nagy Pál (Ker: 19) Tel: +36 30
Email: nagy.pal7@email.hu


Válasz
Sikeres adatfeltöltés!
OK


Válasz
Sikertelen adatfeltöltés!
OK


Nagy Pál
Név:
Nagy Pál
Tel:
+36 30 164 4377
Email:
nagy.pal7@email.hu
Ker:
19
Módosítás


Válasz
Sikeres adatfeltöltés!
OK


Válasz
Sikertelen adatfeltöltés!
OK

Reszponzív viselkedésű weboldal készítés és formázás

10 pont

Adott az index.html, style.css, kutya.avif állományok!

Módosítások az index.html állományban:

1. Hivatkozzon a style.css állományra!
2. Hozzon létre a navigációnak a megfelelő szelektorral egy részt a header alatt!
    a. Tegyen bele három hivatkozást a minta alapján!
    b. Ezek a hivatkozások majd mutassanak a tartalmi rész megfelelő pontjára! Ha rákattintunk oldalon belül navigáljon oda!
3. Hozzon létre a tartalmi résznek egy container osztályjelölővel ellátott egységet!
    a. Szelektálja 3 egységre a megfelelő szelektorokkal!
    b. Mindhárom egység Azonosítója legyen a hozzá rendelt cím (Minta alapján).
    c. A szolgáltatás szekció alatt jelenjen meg egy vonal!
    d. A címe 2-es címsorok legyenek!
4. Szolgáltatásaink szekció:
    a. A cím után lévő bekezdés után jelenjen meg a kutya.avif kép! Ha nem tudná betölteni írja ki a „Vizsgálóasztalon egy kutya” szöveget! Majd lássa el hero-img osztályjelölővel!
    b. A kép alatt hozzon légre egy szelektort services-grid osztályjal.
    c. Ezen belül hozzon létre három elemet service-card osztályjelölővel.
        i. a címek 3 szintű címsorok majd egy bekezdés!
5. Az Árlista szekció:
    a. Készítse el a táblázatot a mint alapján. Figyeljen a tábla kiemelt részére! Két fő részre van osztva!
6. Kapcsolat szekció:
    a. Készítse el a minta alapján!

Állítsa be a következő stílusokat a megadott szelektorokra:

services-grid osztályjelőlőre:

- Ha a benne lévő elemek nem férnek el egymás mellett legyen <u>rugalmas</u>, akkor tördelje őket több sorba!
- A benne lévő elemek közötti távolságot állítsa 20px-re.

- A tördelt elemek kerüljenek középre!
- A külső margót alul felül 30px oldal 0 px legyen.

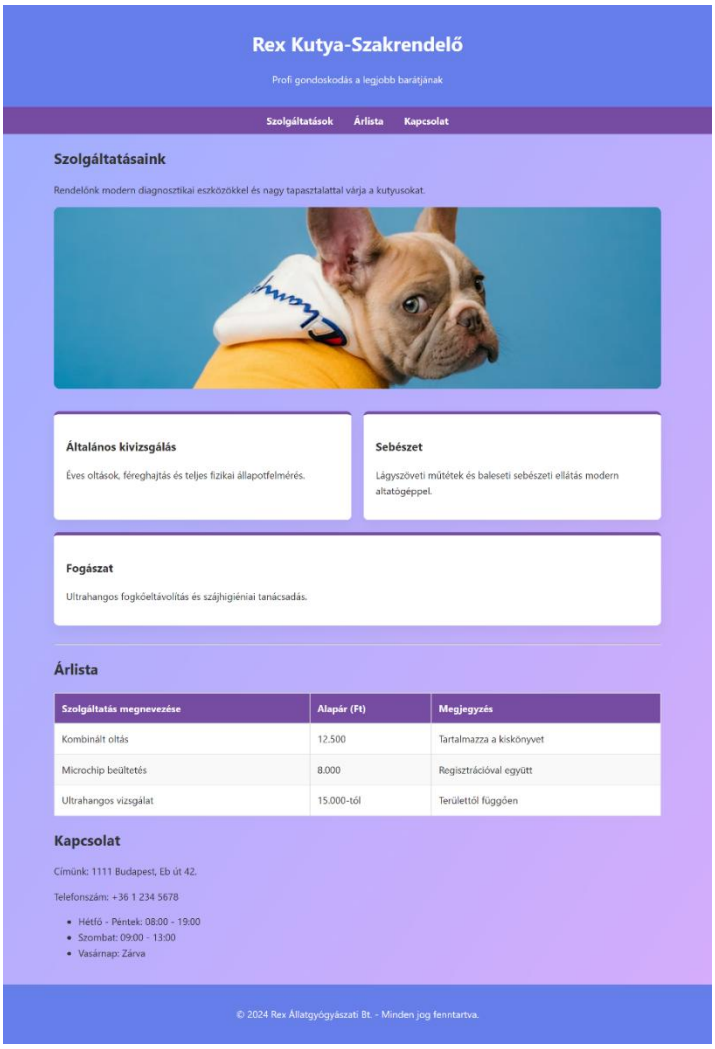
service-card osztályjelölőre:

- felső szegély legyen 5 px folyamatos és #764ba2 színű
- árnyékolás a dobozon a jobb alsó sarok felé (az értékeket szabadon választhatja)
- kerekítse a szegélyeket 8px-re.

Felhasználható dokumentációk:

https://www.w3schools.com/

Minta: (screenshot.png)



Backend programozás és tesztelés

15 pont

Indítsd el az xampp alkalmazás segítségével az `apache` és `mysql` szervereket.

Az `npm start` szkript futtatása után a http//localhost:3000 címen fut az alkalmazásunk.

A következő dokumentációkat használhatja a vizsga alatt:

https://sequelize.org/

https://expressjs.com/

https://jestjs.io/

1. Feladat

Állítsd be a következő útvonalakat a `dogRoutes.js` fileban:

- /range
  - HTTP Metódus: `GET`
  - Controller metódusa: `getDogsByAgeRange`
- /bulk
  - HTTP Metódus: `POST`
  - Controller metódusa: `bulkCreateDogs`

2. Feladat

Készítsd el az egyes feladatban használt metódusokat a `dogController.js` fileban.
Kettő paraméter fog érkezni a /range endpointon a kérés `queryjén` keresztül:

1. minAge: a keresett kutyák minimális életkora
2. maxAge: opcionális paraméter: a keresett kutyák maximális életkora

Használd a `dogService`-ben megírt metódusokat!

3. Feladat

Készítsd el a `getDogsByAgeRange` és `bulkCreateDogs` nevű metódusokat a `dogRepository.js` fileban.
Használd a már létező hibaosztályokat megfelelő hibakezeléshez a tanult módon.

A következőképpen működnek a metódusok:

`getDogsByAgeRange(minAge, maxAge):`
- Visszaadja a kutyák egy olyan részhalmazát, ami eleget tesz a paraméterben érkezett feltételeknek.
- A `maxAge` legyen opcionális paraméter.

`Példák:`

fn(10) = azokat a kutyákat adja vissza, amik legalább 10 évesek

fn(3, 5) = azokat a kutyákat adja vissza, amik legalább 3 és legfeljebb 5 évesek

`bulkCreateDogs(dogs):`
- A paraméterből kapott dogs objektum összes objektumát rögzítse az adatbázisban

4. Feladat

A `getDogsByAgeRange` metódust egészítsd ki úgy, hogy a keresett részhalmazban lévő kutyákhoz hozzárendeljük a gazdájuk adatait.

A tanult módszerek alapján töltsd be a kapcsolódó `Gazda` modell adatait.

5. Feladat

Készítsd el a controllerben található 2 metódust a dogService.js fileban.

Következőket vizsgáljuk első sorban a kérések feldolgozásánál:

`getDogsByAgeRange:`
- a minAge nem lehet nagyobb a maxAge paraméternél. Dobj ValidationError-t, ha ez

`bulkCreateDogs(dogs):`
- Validáld le a paraméterben érkezett tömb tartalmát a következőképpen:
    - A dogs objektum tömb típusú kell, hogy legyen. Ha nem tömb, akkor dobjunk ValidationErrort megfelelő hibaüzenettel.
    - A dogs objektumban olyan objektumok tartalmazhatnak, amikben a Kutya modell kötelező attribútumai szerepelnek. Ha bármelyik kötelező attribútum hiányzik, akkor dobjunk ValidationErrort megfelelő hibaüzenettel.

6. Feladat

Egészítsd ki az `dog.api.test.js` file-t.

A tesztesetek és teszt-csoportok leírását készítsétek a tanult módon és konvenciók alapján.

Teszteld le a `supertest` csomag segítségével mindkettő endpointodat.

Az adatokat beégetheted a tesztesetekbe.

Figyelj arra, hogy a tesztkörnyezetben bármilyen módosítást végző adatbázis műveletet igénylő művelet `tranzakciót` használ és a kérés végén visszaáll a kérés előtti állapotra.

Ennek figyelmében készítsd el a megfelelő teszteket.

Frontend programozás - 15 pont

Adott egy előre megírt, de részben hiányos react alkalmazás. Feladata, hogy kiegészítse néhány a feladatban megadott funkcióval, komponenssel! Kihasználhatja a már megírt kódokat!

A macskak/index.tsx állományban hiányozik három doboz. Ehhez hozza létre a következőket!

1. Hozzon létre egy új fájlt egy új komponenst MacskaCard néven!
    a. Adja meg a fő elemének a „cat-card” nevű osztályt!
    b. Ennek a fő elemnek a tartalma a MacskaCard felhasználása során beírt utódok tartalma legyen!
    c. Tegye elérhetővé más komponensek felhasználáshoz!
2. Hozzon létre egy új fájlt egy új komponensnek MacskaKartyak néven!
    a. Amikor a komponenst használjuk lehessen átadni az osztály nevét és egy attributomot értékkel (Lásd a macskak/index.tsx fájlban)
    b. Az átadott paraméterek legyen beállítva ennek a komponensnek a fő szelektorába!
    c. Majd ebben hozzon létre a három MacskaCard komponenst! (Ha nem sikerült a MacskaCard komponenst létrehoznia, vagy alkalmaznia, használjon diveket! Ebben az esetben csak részpont adható)

A kutyak/index.tsx lehetővé teszi az adatok megjelenítését! (Lásd minta) Továbbá van két legördülő lista a Műveleteknek (Módosítása, Új elem hozzáadása) és a Fajtáknak (Válassz(létezők))!

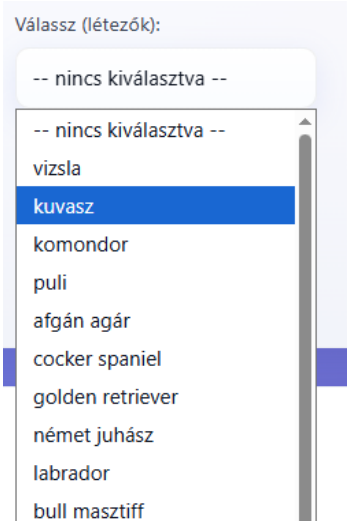
3. A már lekért adatok segítségével jelenítse meg a Válassz résznél a kutyák fajtáját! Kerüljön bele az összes (az adatbázisban lévő összes) fajta!
4. A műveletek kiválasztásánál:
    a. Új adat:
        i. Ebben az esetben Ne látszódjon a fajták (legördülő lista)!
        ii. Az input mezők alatti gomb neve legyen „Új kutya hozzáadása”
        iii. Amikor az input mezőket kitöltjük, erre a gombra kattintva küldjük fel a backend (api) segítségével az adatokat! (http://localhost:3001/api/species)
        iv. A Kép URL résznél használhatja ezeket a képeket: https://placedog.net/500/280
    b. Módosítás:

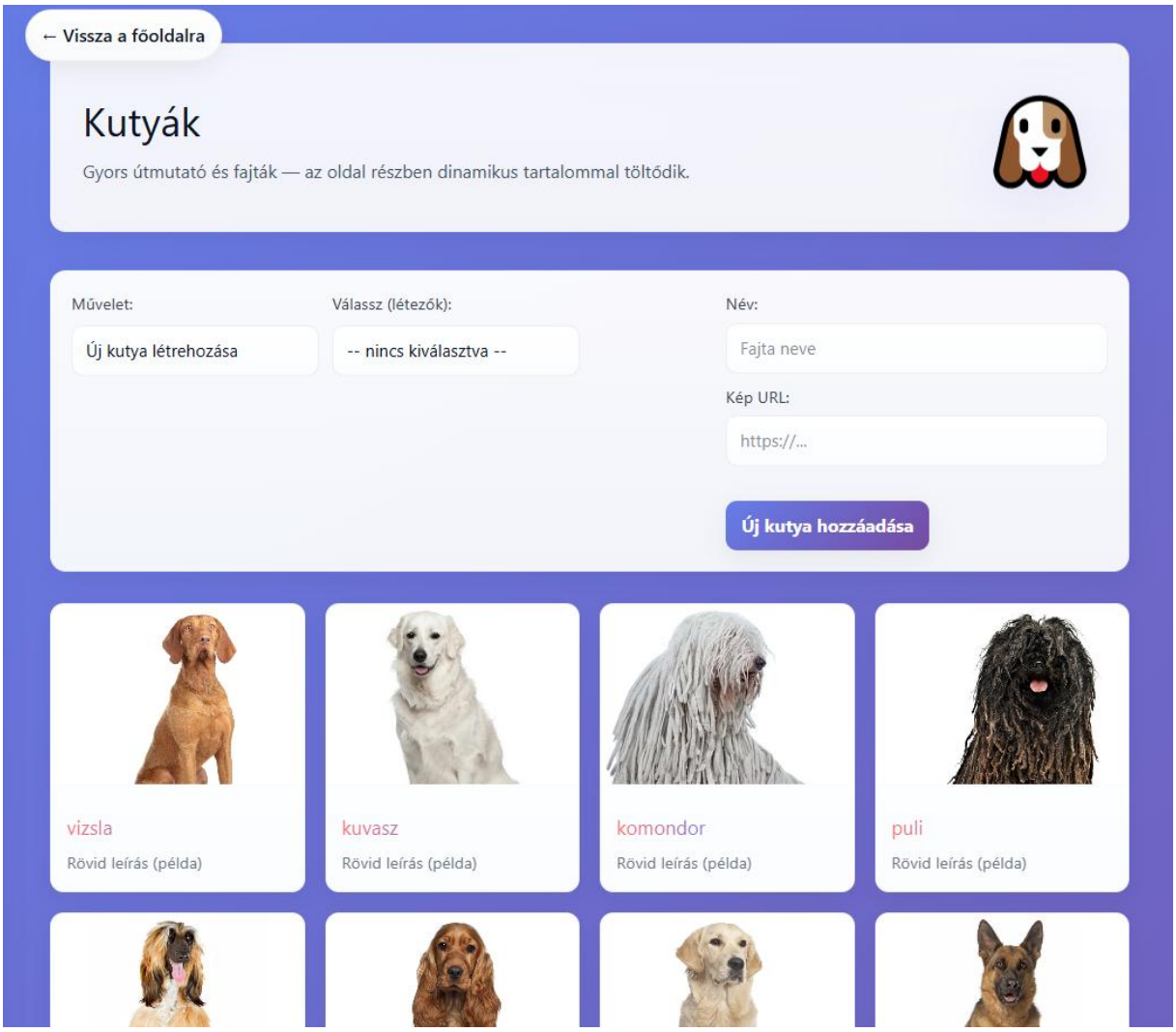
i. Látszódjon a fajták legördülő listája!
ii. Lehessen kiválasztani egy fajtát!
iii. Ebben az esetben kerüljenek a fajtához tartozó információk az input mezőbe, hogy a felhasználó maga döntse el melyiket szeretné módosítani!
iv. Az inputok alatti gomb megnevezése változzon „Módosíts mentése” feliratra!
v. Erre gombra kattintva módosuljanak az adatok!

Használható dokumentációk:

https://tanstack.com/
https://axios-http.com/
https://ui.shadcn.com/
https://www.w3schools.com/

Minta:


Válassz (létezők):
-- nincs kiválasztva --
-- nincs kiválasztva --
vizsla
kuvasz
komondor
puli
afgán agár
cocker spaniel
golden retriever
német juhász
labrador
bull masztiff


← Vissza a főoldalra
Kutyák
Gyors útmutató és fajták — az oldal részben dinamikus tartalommal töltődik.
Művelet:
Válassz (létezők):
Név:
Új kutya létrehozása
-- nincs kiválasztva --
Fajta neve
Kép URL:
https://...
Új kutya hozzáadása
vizsla
Rövid leírás (példa)
kuvasz
Rövid leírás (példa)
komondor
Rövid leírás (példa)
puli
Rövid leírás (példa)